INTRODUCTION I: FOUNDATIONS OF PROGRAMMING

PROGRAMMING LANGUAGE-INDEPENDENT CONCEPTS

2025-10-02 14:42

WHAT IS »PROGRAMMING«?

Describing the solution to a problem in a sufficiently precise way that it can be executed by a computer.

A program is

- a sequence of commands or instructions that
- processes a set of data.

Niklaus Wirth:

ALGORITHMS + DATA STRUCTURES = PROGRAMS

© MH

A FIRST PROBLEM

THE »NIM« GAME

From a pile of matches, two players take turns to take away 1 to (for example) 3 matches, the one who takes **the last match** being the winner.

Program invocation:

```
python3 NIM.py
```

```
We play with 17 matches – 1-3 must be taken away at a time.
I start.
||||| ||||| ||||| ||
I take 1 matches.
||||| ||||| ||||| |
Your move (1-3): 3
||||| ||||| |||
I take 1 matches.
||||| ||||| ||
Your move (1-3): 3
||||| ||||
I take 1 matches.
||||| |||
Your move (1-3): 3
|||||
I take 1 matches.
||||
Your move (1-3): 3
|
I take 1 matches.
I won.
```

WE WILL LEARN TO UNDERSTAND THE PROGRAM IN THE NEXT COURSE UNITS.

© MH

WHAT HAPPENS?

IN PRINCIPLE

Interpreted languages (Python, Java, …)**:**

1. The compiler analyzes the source code (*.py) and compiles it into an intermediate code (»bytecode«). Bytecode is a low-level, **platform-independent** representation of the source code that a Python interpreter can execute. It is stored in .pyc files in the __pycache__ directory.
2. The **platform-specific** interpreter reads the bytecode and executes it instruction-by-instruction.

Compiled languages (C, C++, …):

1. The compiler analyzes the source code (e.g., *.c) and compiles it into native machine code for the target platform (»object code« *.o).
2. The linkage editor augments this code by binding external references (to program libraries etc.), creating a complete machine program (a.out).
3. The loader copies this program into main memory and transfers the CPU control to the entry (starting) point of the program.

© MH

WHAT HAPPENS?

COMPILATION VS. INTERPRETATION

Example

You find a **Roman recipe** and want to try it out…

Solution I (compiler):

- We ask a **Roman** to translate the recipe into English
- We give the translated recipe to a **cook** (≙ CPU)

Solution II (interpreter):

- We find a **Roman who can cook** and cooks the dish himself

Variations

We distinguish **three classes** of programming languages:

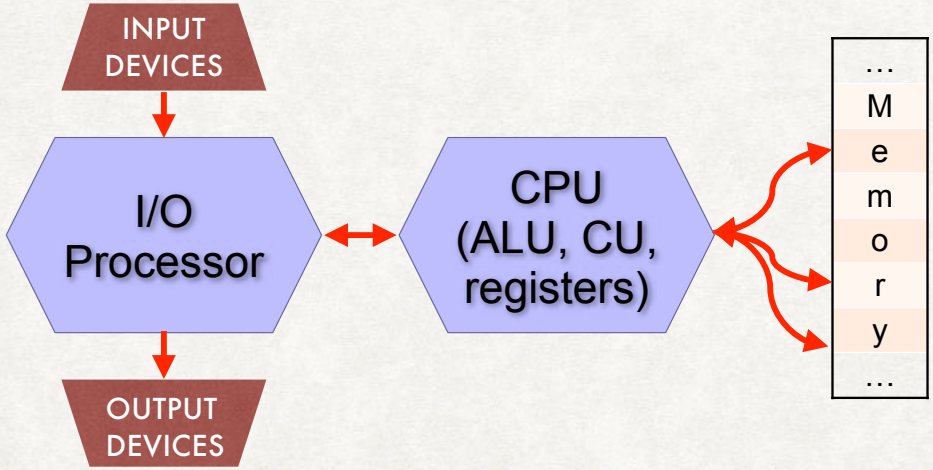
1) Compiled (C, C++, …)
2) Interpreted (PHP, Js, …)
3) Hybrid (Java, Python, …): They are **compiled** (i.e., translated) into an intermediate representation (byte code) which is then **interpreted** by an interpreter

© MH

DATA

A VERTICAL TABLE OF BYTES (WHERE EACH ROW MAY CONSIST OF SO MANY BYTES THAT THE VALUE SHOWN CAN BE STORED IN IT)

The data that we *process* is located in the so-called main memory and is accessed (read or written) by the central processing unit (CPU), consisting of the arithmetic-logic unit (ALU), the control unit (CU), and registers (fast memory cells):



The data are binary coded (0/1). Depending on the coding rule we can store integers, rational numbers, letters, etc.

Individual cells of the main or random access memory (RAM) are addressed as variables in the program.

Our simplified memory model:

...
4711
3.1415926
0
'M'
'H'
...

(The commands are also stored in the main memory → »von Neumann architecture«)

John von Neumann: First Draft of a Report on the EDVAC (1945). In: IEEE Annals of the History of Computing. Vol. 15, Issue 4, 1993

© MH

VARIABLES

IN HIGHER PROGRAMMING LANGUAGES

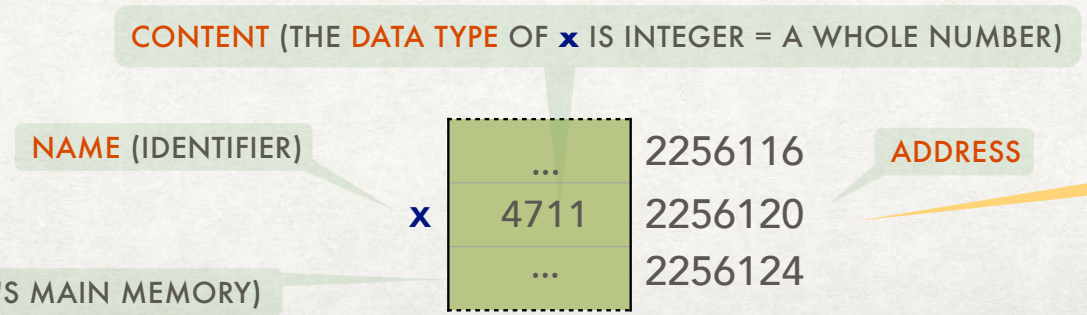
Variable

is a **named** »**data container**«, with

- address (position in memory) – corresponds to its identifier (name);
- type or data type (category) – defines the allowed **values**, the **size** in memory, as well as the supported **operations**;
- value (content) – may change during the course of the program (»**at run time**«). It is **binary** coded: $17_{10} = 10001_2$ – consisting of bits, bytes (8 bits), words (2 - 8 bytes)

Memory model (»classical«, non-Python)

The **Python** memory model will be introduced soon.



RIDDLE
WHAT IS THE SIZE OF VARIABLE **x** (IN BYTES), IF THIS IS THE MEMORY OF A COMPUTER WITH 1-BYTE MACHINE WORDS?

© MH

RELATIVES OF VARIABLES

IN HIGHER PROGRAMMING LANGUAGES

Literal

is a data value that appears in the program text without a variable name (but may be »packed« into an invisible variable by the compiler),
e.g., a number: **5** or **3.6**, or a text: "This is a text literal".

Constant

is a variable whose content cannot be changed - e.g., **PI** for 3.1415926535897 93...

Other features are shared with the notion of a variable
(address, data type, name – for constants).

© MH

DATA TYPE

All data have a type (»are of type …«, e.g., »integers«), which

- defines the **interpretation** of the code stored in memory:

  AN ELEMENT OF A DATA TYPE (E.G., 5) IS CALLED A VALUE OR AN INSTANCE. CREATING A DATA ELEMENT OF A GIVEN TYPE IS CALLED INSTANTIATING (THE TYPE).

- defines the value range, e.g., -32768 .. 32767
- … and thus, the memory requirements (e.g., 2 bytes)
- defines the allowed operations, e.g., **+**, **-**, *****, **/**, **=**, **<**, **>**
- restricts the use of the data:

  x + "A" (forbidden because meaningless, if the type of **x** is numeric)

  → Helps to avoid erroneous nonsensical use → Type safety

CONTENT SHOWN IN BINARY.
DATA TYPE OF **x**: INTEGER
DATA TYPE OF **y**: CHARACTER

WHOLE NUMBER: 67

CHARACTER: C

	…	4122312
x	0L0000LL	4122313
y	0L0000LL	4122314
	…	…

In **Python**, the data type of a variable **can change** in the course of the program.
In many **other languages** (Java, C, C++ …, especially the compiled languages),
the data type of a variable **is fixed** and must be specified at the point of definition.

© MH

ALGORITHM

:= Precise, step-by-step instructions for solving a problem

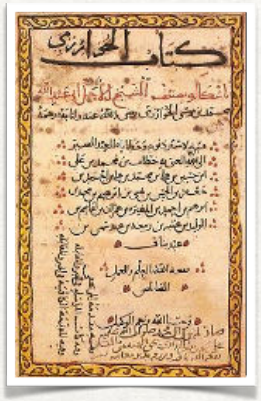
CF. ASSEMBLY INSTRUCTIONS FOR DO-IT-YOURSELF FURNITURE, COOKING RECIPES, DIRECTIONS FROM A ROUTE PLANNER...

Etymology

Muhammad ibn Musa Al-Chwarizmi (latinized *Algorismi*)
780-859, astronomer and mathematician from Baghdad.

825 A.D.: First textbook on the rules of arithmetic using the decimal system originating in India (basic arithmetic operations).

→ »Algorithm«



ALGORITHMS + DATA STRUCTURES = PROGRAMS

https:#de.wikipedia.org/wiki/Al-Chwarizmi

© MH

DIFFERENT SOURCES COME UP WITH DIFFERENT PROPERTY SETS...
C.f. https://en.wikipedia.org/wiki/Algorithm_characterizations

ALGORITHM

PROPERTIES

Properties that a procedural description **should satisfy to be called** an algorithm:

- An algorithm must be unambiguously describable in a finite text (finiteness). This description is based on a certain »processor type« (human, von Neumann computer, register machine, Turing machine, quantum computer, ...).
- Each step must actually be executable on that »processor type« (executability).
- It may only require finite memory at any time (dynamic finiteness).
- It may only require a finite number of steps (termination).

In practice, there are often **additional restrictions**:

- The algorithm must produce the same result under the same conditions (determinacy).
- The next step to be applied must uniquely defined at each point in time (determinism).

© MH

INSTRUCTIONS AND STATEMENTS

Instructions control the processor – the **instruction set** for machine programs **depends on the processor type**, e.g. (here: *hypothetical* machine language):

LDA 10 — LOAD 10 INTO THE ACCUMULATOR REGISTER

MUL 6 — MULTIPLY THE CONTENT OF THE ACCUMULATOR REGISTER BY 6

ADD 7 — INCREASE THE CONTENT OF THE ACCUMULATOR REGISTER BY 7

STA x — STORE THE CONTENT OF THE ACCUMULATOR REGISTER IN THE MEMORY CELL WITH ADDRESS X

$x \leftarrow 10 * 6 + 7$

In a high-level programming language, the instructions (statements, commands) are **independent of the processor type**. The most important ones are:

- Evaluation of expressions (calculations) and value assignment (e.g. **x ← 10 * 6 + 7**)
- Command sequence* (compound statement, composite statement, sequence)
- Branching* (conditional statement, selection, choice) – two-way or multiway
- Loop* (iteration, repetition) – head-controlled or foot-controlled
- Subroutine call
- ~~Jump statement~~

*BÖHM & JACOPINI (1966): „EVERYTHING THAT IS IN PRINCIPLE COMPUTABLE CAN BE COMPUTED WITH COMMAND SEQUENCE, (TWO-WAY) BRANCHING AND REPETITION."

Corrado Böhm & Giuseppe Jacopini: Flow diagrams, turing machines and languages with only two formation rules. CACM 9, 5 (1966), 366-371

©MH

- Evaluation of expressions (calculations) and value assignment (e.g. x ← 10 * 6 + 7)
- Command sequence* (compound statement, composite statement, sequence)
- Branching* (conditional statement, selection, choice) – two-way or multiway
- Loop* (iteration, repetition) – head-controlled or foot-controlled
- Subroutine call

VALUE ASSIGNMENT

The figure shows the standard model. The **Python** memory model will be introduced soon.

x ← 10 * 6 + 7

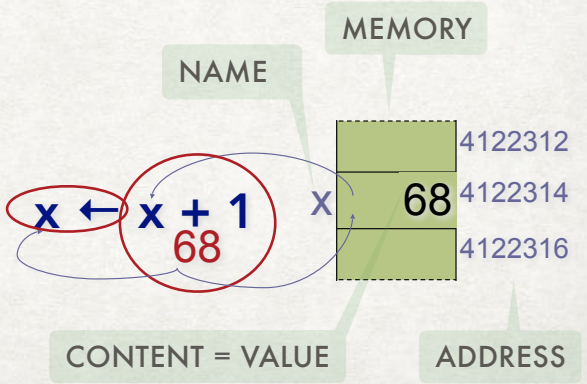
- Assigns the value of the expression on the right side (r-value; to the right of the assignment symbol ←) to the variable on the left side (l-value, here: **x**) after its evaluation (here: 67).
  The variable stores the value until the next assignment.
- An earlier content of the variable is **overwritten** by the assignment.
- The variable can be used in any calculations after the assignment.

Python: **x = 10 * 6 + 7**

Pascal: **x := 10 * 6 + 7;**

x ← x + 1

- Changes the value of the variable **x**
- Adds 1 to the previous value
- Assigns the result to **x** again



© MH

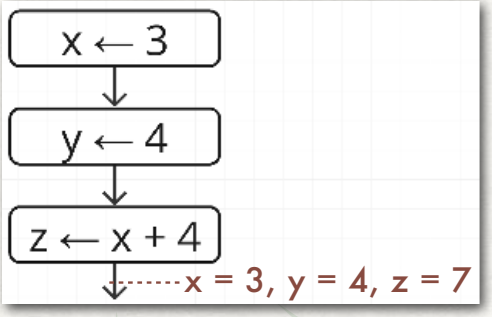
- Evaluation of expressions (calculations) and value assignment (e.g. x ← 10 * 6 + 7)
- Command sequence* (compound statement, composite statement, sequence)
- Branching* (conditional statement, selection, choice) – two-way or multiway
- Loop* (iteration, repetition) – head-controlled or foot-controlled
- Subroutine call

COMMAND SEQUENCE

Consecutive execution of several statements

Statement 1
Statement 2
…
Statement k

x ← 3
y ← 4
z ← x + y



GRAPHICAL NOTATION:
NASSI-SHNEIDERMAN DIAGRAM
(STRUCTOGRAM)
SIMPLE RECTANGULAR INSTRUCTION BLOCKS, NESTED IF NECESSARY

GRAPHICAL NOTATION:
FLOW CHART (FLOW DIAGRAM)
STATEMENTS ARE CONNECTED BY ARROWS REPRESENTING THE FLOW OF CONTROL

OPTIONAL ASSERTION:
STATEMENT **ABOUT** THE STATE OF THE ALGORITHM AT A CERTAIN POINT

Command sequences also allow their statements to be logically combined into a unit that can be treated **as a single statement** in a greater context.

© MH

- Evaluation of expressions (calculations) and value assignment (e.g. x ← 10 * 6 + 7)
- Command sequence* (compound statement, composite statement, sequence)
- Branching* (conditional statement, selection, choice) – two-way or multiway
- Loop* (iteration, repetition) – head-controlled or foot-controlled
- Subroutine call

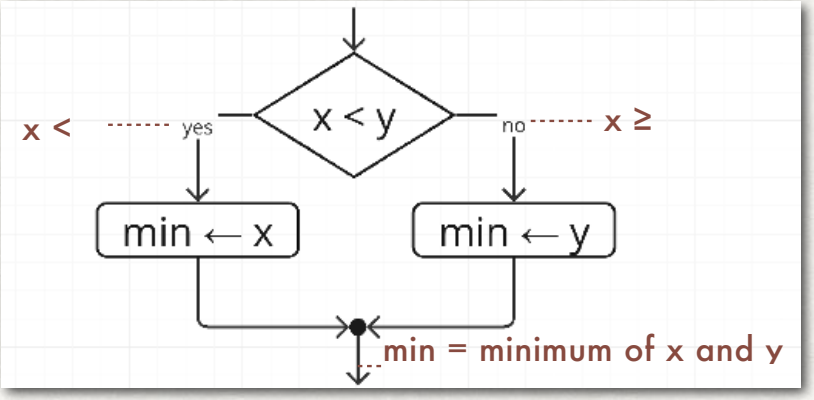
BRANCHINGS

Two-way branching

Selection of the next statement(s) based on a logical expression

Condition?	
yes	no
Statement_{yes}	Statement_{no}

x < y ?	
yes	no
min ← x	min ← y



Multiway branching

Selection of the next statement(s) based on an expression with a finite value range

value = ?			
1st value	…	…	otherwise
Statement_1	$\text{Statement}_{\ldots}$	Statement_n	$\text{Statement}_{otherwise}$

© MH

- Evaluation of expressions (calculations) and value assignment (e.g. x ← 10 * 6 + 7)
- Command sequence* (compound statement, composite statement, sequence)
- Branching* (conditional statement, selection, choice) – two-way or multiway
- Loop* (iteration, repetition) – head-controlled or foot-controlled
- Subroutine call

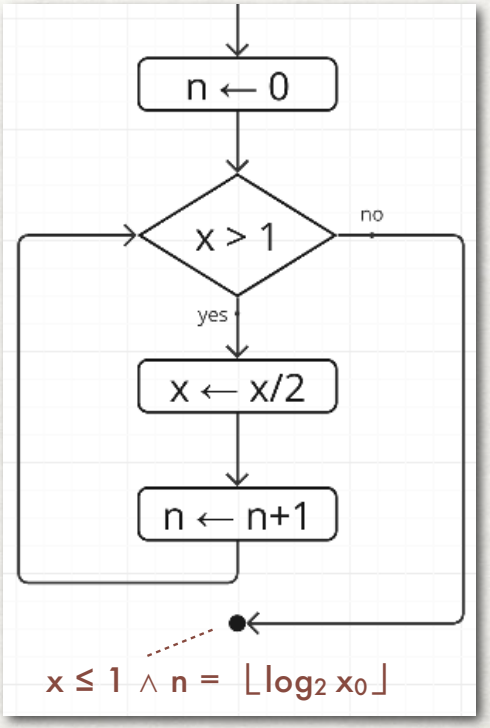
REPETITION

Head-controlled loop

Repeat as long as a condition checked at the **loop entry** is fulfilled

```
While …
    Statement
```

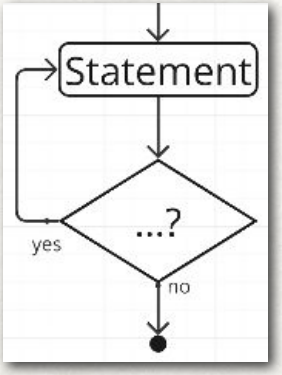
```
n ← 0
While x > 1
    x ← x / 2
    n ← n + 1
```



Foot-controlled loop

Repeat as long as a condition checked at the **loop exit** is fulfilled

```
    Statement
While …
```

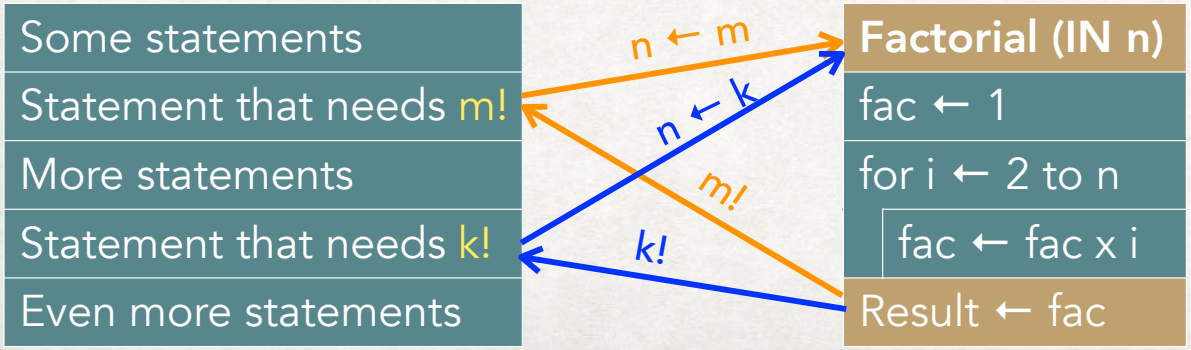


© MH

SUBROUTINES

Parts of a program can also be formulated elsewhere as a subroutine (subprogram), where the control flow

- branches there if required (call), possibly transporting data (arguments) into the subroutine and
- returns to the place of the call after the subroutine has terminated (return), possibly yielding a result



We use subroutines to

- factor out common parts of an algorithm for reuse
- structure larger programs in »digestible units« (modules)

Some **special types** of subroutines:

- Functions: Compute a value from given arguments (like **Factorial(n)**)
- Procedures: Produce a certain effect (e.g., print something on the screen)
- Methods: Used to implement operations on data types, e.g., **"John F. Kennedy".find(' ')** (→ 4)

TEXT STRING

CALL OF METHOD **find()** FOR THAT TEXT STRING

IN PYTHON ALSO CALLED »FUNCTION«

© MH

»NIM« GAME

PART 1: PREPARATION

VARIABLE

LITERAL

COMMENT

matches ← 17	# e.g.
max_take ← 3	# e.g.
"We play …"	# Output
comp_turn ← Random(True, False)	# »Flip coin«

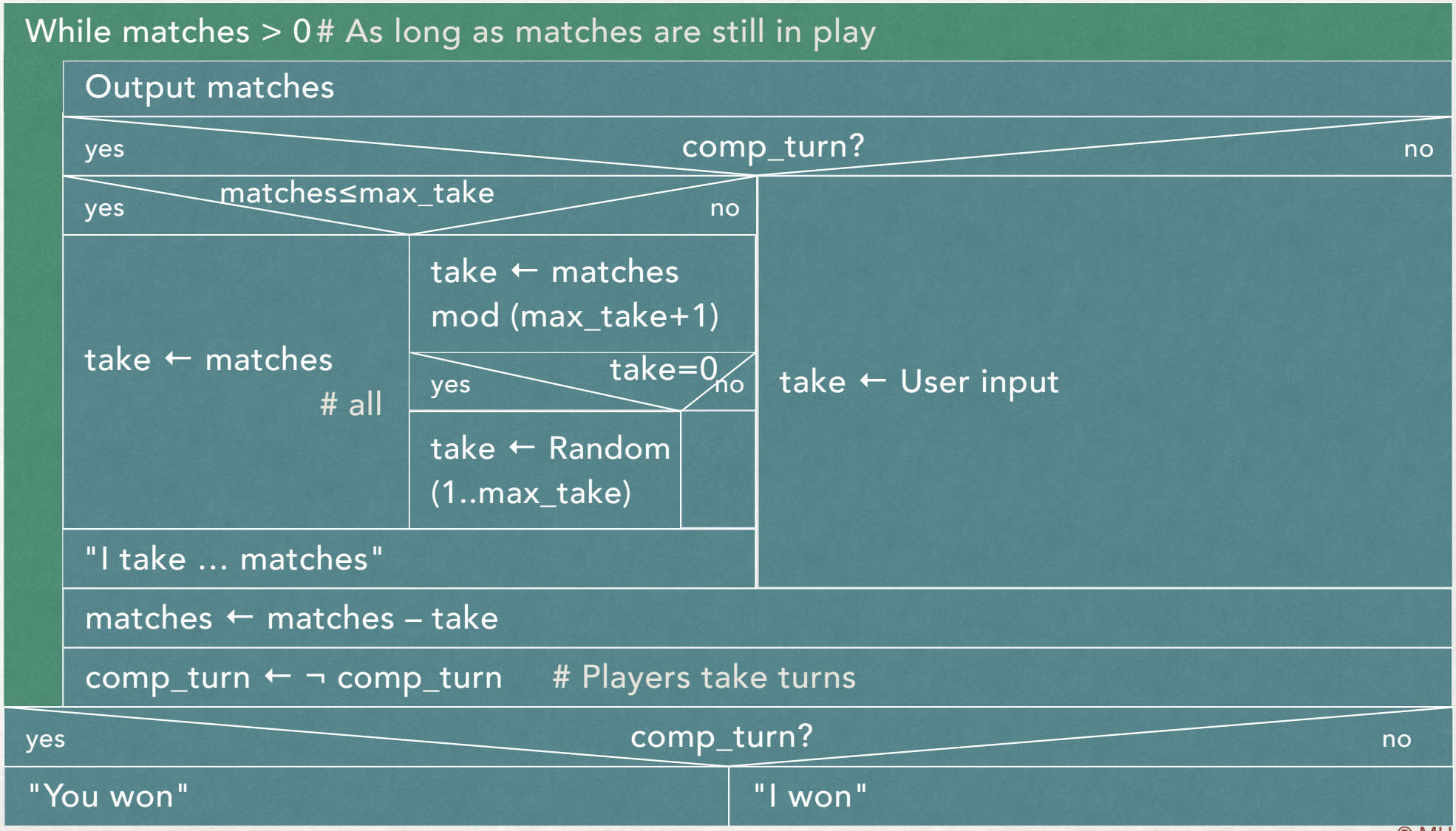
yes	comp_turn?	no
"I start"		"You start"

SHORT FOR THE COMMAND "OUTPUT 'I START' ON THE CONSOLE WINDOW"

© MH

»NIM« GAME

PART 2: GAME



© MH

THE END